康希诺生物数字化落地方案

AI 驱动研发效能提升

康希诺生物股份公司
2026.9

数字化研发蓝图议程

01 AI 助力研发流程

02 研发支撑体系建设

03 AI 能力平台与场景

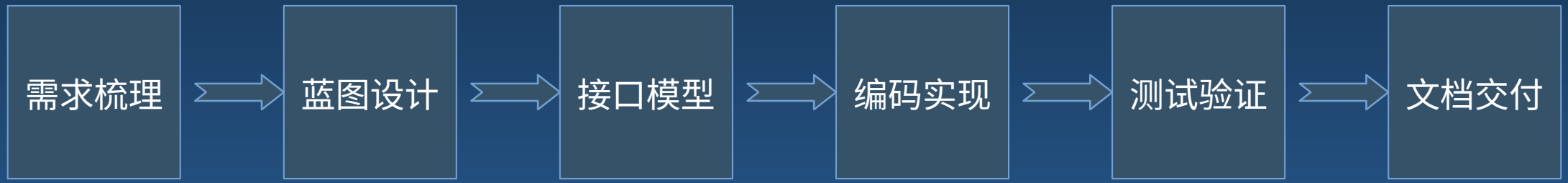
04 实施路径与保障

AI 助力研发流程

一个新需求，即便初始输入仅是模糊的业务描述，也可依托 AI 沿

「需求梳理 → 蓝图设计 → 接口模型 → 编码实现 → 测试验证 → 文档交付」

六个环节逐步推进，直至完成交付。



AI 助力研发流程 *需求梳理*

BEFORE · 传统模式痛点

· 业务方提出需求（如 " 希望查询批签发状态 "）后，开发、业务、测试需多轮沟通仍难以确认口径，需求确认周期常以周计。
· 用户故事与验收标准依赖产品经理人工撰写，遗漏条款通常在测试阶段才暴露。
· 同一需求在业务、开发、测试三方理解不一致，缺乏对齐机制，分歧在交付验收时才显现。
· 同类功能调研依赖人工检索，效率低且覆盖不全。

AFTER · AI 智能赋能升级

· **AI 需求澄清**：将业务原始诉求输入 AI ，由其按目标、约束、边界、异常处理四个维度生成澄清问题，短期内收敛为字段完整、口径统一的结构化需求单，人工仅做确认审定。
· **用户故事与验收标准生成**： Given-When-Then 格式的初稿由 AI 产出，产品经理的工作由起草转向审核，遗漏项由模板机制兜底。
· **需求单一事实源**：结构化需求单作为三方对齐的唯一依据，模糊表述均要求 AI 给出定义与示例，理解偏差在评审前即暴露并消除。
· **AI 辅助调研**：同类功能的实现方式与常见问题由 AI 汇总为要点清单，调研效率显著提升。

模糊需求一键结构化，实现全流程效率跃升
告别低效的人工反复确认， AI 让需求梳理从“人工堆砌”转向“智能智造”，大幅降低沟通成本与返工率。

50%+
整体需求处理效率提升

AI 助力研发流程 *蓝图设计*

BEFORE · 传统模式痛点

· 架构方案依赖少数资深工程师，其时间投入成为项目瓶颈；方案深度受限于个人经验边界。
· 技术选型缺乏系统评估，多以既往使用经验为依据，契合度、生态、学习曲线与长期维护成本难以横向比较。
· 技术风险（单点故障、数据一致性、性能瓶颈等）多在编码过半时暴露，返工代价最高。
· 方案对比与决策材料依赖人工整理，编写周期长，评审效率低。
· GMP 数据完整性、等级保护、数据安全法等合规要求需事后逐项咨询确认，架构设计与合规要求脱节。

AFTER · AI 智能赋能升级

· **多方案生成**：基于结构化需求， AI 输出单体、微服务、事件驱动等多种候选架构，并分别说明适用条件与代价；团队工作重心转为方案评估与组合优化。
· **技术选型对比矩阵**： AI 按性能、生态成熟度、团队熟悉度、合规契合、维护成本等维度生成对比评分表，每项选择均有依据、可答辩。
· **对抗式风险评审**： AI 以运维、安全、审计等多视角对方案进行审查，风险清单在编码开始前形成；风险前置是蓝图环节的核心收益。
· **决策记录自动生成**：对比矩阵与架构决策记录（ADR）初稿由 AI 完成，评审会由 " 听取汇报 " 转变为 " 直接决策 " 。
· **合规约束预置**：将 GMP 、数据分级等要求作为约束输入 AI 分析上下文，架构设计自初始阶段即包含合规视角。

核心收益：架构设计效率提升 **40%+** ，实现从“经验驱动”到“数据智能驱动”的范式转变，将技术风险前置化解，大幅降低后期维护成本。

AI 助力研发流程 *接口模型*

BEFORE · 传统模式痛点

· PRD 与接口设计依赖人工起草，字段级细节易遗漏，问题集中在联调阶段暴露。
· 命名、错误码、分页等风格各模块不统一，接口对齐依赖口头沟通与零散记录。
· 数据模型多次变更后，文档、代码、库表三者不同步，以文档还是以代码为准难以界定。

AFTER · AI 智能赋能升级

· **契约生成**： AI 基于结构化需求与 PRD 生成字段级接口定义（OpenAPI）与数据模型建议，评审对象由 " 起草物 " 变为 " 修订物 " 。
· **团队规范内置**：将命名、错误码、分页规范作为约束提供给 AI，生成结果即符合规范，新增接口与存量接口保持同构。
· **契约即事实源**：模型变更由 AI 同步更新契约与文档，前后端基于同一份契约并行开发，联调为验证过程而非发现过程。

联调阶段接口类问题前移至设计阶段解决
先立契约、再动工。接口契约在 AI 辅助下前置确定，跨团队协作的不确定性显著下降。

AI 助力研发流程 *编码实现*

BEFORE · 传统模式痛点

- 样板代码与重复性 CRUD 开发占据主要工时，资深工程师精力消耗于机械性编写而非设计。
- 新框架、新语言上手周期长，新成员数周内难以独立提交，资深成员亦常被陌生 API 排查阻塞。
- 遗留代码缺乏文档、命名晦涩，理解历史实现方式成本高，改动风险大，无人维护的模块持续累积。
- 代码风格因人而异，评审精力被格式、命名等基础问题消耗，逻辑与设计问题反而缺乏审查深度。
- 开发过程中频繁中断以查阅资料、编写调试脚本，工作连续性差。

AFTER · AI 智能赋能升级

- **代码框架生成**： AI 基于接口契约生成模块骨架与重复性代码，开发者专注核心业务逻辑的填充与逐行审查，精力回归设计本身。
- **上下文感知补全**： IDE 内的 AI 补全理解整个工程上下文，新成员即可产出符合项目惯例的代码，技术栈上手周期显著压缩。
- **代码解释**：遗留模块由 AI 逐段解释、补齐注释与文档，理解成本大幅下降，不可维护模块转为可读资产。
- **规范前置**：团队编码规范作为 AI 生成约束，提交前由 AI 完成基础检查，代码评审聚焦逻辑与设计。
- **连续性保障**：资料查询、调试脚本编写、报错解释等中断性动作由 AI 就地完成，开发过程保持连贯。

预期效果：编码效率提升（参考值： 30% 以上），新成员上手周期显著缩短。

AI 助力研发流程 *测试验证*

BEFORE · 传统模式痛点

· 用例设计依赖个人对需求的理解，覆盖以经验为限，边界与异常场景长期缺位。
· 自动化脚本编写耗时、维护成本更高，界面变更后脚本批量失效，团队对自动化投入意愿持续走低。
· 缺陷定位依赖人工检索日志、逐级排查，经验型判断，新成员难以胜任。
· 回归范围依据经验估算，全量回归效率低、缩减范围风险高，发布决策缺乏依据。
· 测试在流程末端介入，问题发现时代码已完成大量开发，修复代价最高。

AFTER · AI 智能赋能升级

· **用例集生成**： AI 直接基于需求单与验收标准生成正常、边界、异常用例，人工补充业务判断；用例覆盖由经验驱动转为系统性穷举。
· **脚本生成与自修复**：接口与 UI 测试脚本由 AI 起草，执行失败时 AI 先行分析并给出修复建议，维护成本降至可持续投入的水平。
· **缺陷智能归因**：日志与调用栈自动聚类，输出根因候选；缺陷定位由人工排查转为 "AI 诊断输入、人工最终确认 " 。
· **精准回归**： AI 基于变更影响分析推荐回归范围，测试效率与发布质量得以兼顾。
· **测试左移**：用例先于代码存在（源于验收标准），开发过程即可依此自测，质量在过程中内建，而非末端拦截。

AI 消除的不是缺陷本身，而是质量对 " 运气与经验 " 的依赖。测试策略制定与风险兜底判断，始终属于团队

用例编写效率提升（参考值： 50% 以上），缺陷平均定位时间缩短。

AI 助力研发流程 *文档交付*

BEFORE · 传统模式痛点

- 文档滞后于代码，文档描述与系统实际行为不一致。
- API 文档手工维护，接口变更后即过期，文档权威性丧失，团队转而直接以源码为准。
- 版本说明与变更日志依赖发布前集中补写，遗漏频发，历史难以回溯。
- 项目知识分散于个人笔记与沟通记录，人员流动即造成知识流失。

AFTER · AI 智能赋能升级

- **文档与代码同源**： AI 基于代码与契约生成并更新 API 文档，随提交同步，文档恢复可信度。
- **手册与指南生成**：用户手册、运维说明由 AI 依据功能描述起草，人工完成审校与定稿。
- **变更日志自动生成**： AI 汇总提交历史自动生成分版本变更说明，发布记录完整可溯。
- **知识自动沉淀**：全过程的需求单、决策记录、评审结论、代码注释由 AI 整理为可检索知识库；交付完成时，项目知识即已留存于组织。

文档随代码自动同步，知识随流程自然留存——人员流动不再等同于资产流失

文档编制效率提升（参考值： 70% 以上），文档与系统行为保持一致。

研发体系建设

CI/CD
流水线

质量门禁

研效度量

问题跟踪

单点提效要成为组织能力，需完成四次 " 固化 "—— 固化进流程、固化为底线、固化为数据、固化为闭环。研发支撑体系即围绕这四件事展开。

研发体系建设 *CI/CD 流水线*

BEFORE · 传统模式痛点

· 构建与部署依赖手工执行或个人维护的脚本，步骤因人而异，难以复现与交接。
· 开发、测试、生产环境配置不一致，问题多在上线环节集中暴露，定位与排查耗时。
· 从代码提交到发布的链路长、反馈慢，问题发现滞后于变更。
· 流水线多靠人工维护，构建失败需人工翻查日志定位，维护成本高。

AFTER · AI 智能赋能升级

· **自动化管道**：代码提交即自动触发构建、测试与部署，全过程标准化、可复现、可追溯，交付不再依赖个别人员的操作。
· **环境即代码**：各环境以配置代码统一管理、一键一致化，环境漂移消除，环境类问题提前暴露。
· **快速反馈**：提交后分钟级获得构建与测试结果，问题锁定到最近一次变更，定位成本大幅下降。
· AI 辅助维护： AI 辅助生成与更新流水线定义，构建失败自动归因并给出修复建议，流水线维护成本显著下降。

支撑体系的第一层价值，是把 " 人会做 " 变成 " 管道自动做 " ，让提效可复制、可交接、不因人员变动而失效

交付链路自动化率提升，发布周期缩短，因人工操作引入的差错显著减少。

研发体系建设 *质量门禁*

BEFORE · 传统模式痛点

· 质量保障依赖代码评审与个人自觉，规范执行因人而异，缺少强制性卡点。
· 检查多发生在流程后段，问题发现晚，修复成本高。
· 静态检查、测试覆盖、代码评审相互割裂，缺乏统一的合入与上线标准。
· 门禁规则若存在，多为静态阈值，难以识别结构性与语义层面的问题。

AFTER · AI 智能赋能升级

· **内建质量**：规范检查、单元测试、覆盖率、 AI 代码审查等纳入合并前自动检查，不达标即拦截，质量不再依赖自觉。
· **卡点前置**：门禁设于代码合并与发版入口，问题在进入主干前即被发现，修复成本最低。
· **统一标准**：质量规则集中配置、全局一致，各团队执行同一套底线标准，结果可度量、可追溯。
· **AI 增强门禁**： AI 审查识别静态规则难以覆盖的设计与安全隐患，门禁随模型迭代持续进化。

质量不是检查出来的，而是由机制兜底的——＂上不了线＂本身就是最有效的规范

缺陷在合入前被拦截，线上问题率下降，质量规则全局一致、可追溯。

研发体系建设 *研效度量*

BEFORE · 传统模式痛点

· 各环节周期、交付吞吐、缺陷水平缺乏统一采集口径，现状难以量化。
· 研发过程数据分散在多个工具中，人工汇总滞后，难以反映真实瓶颈。
· 改进措施多凭经验判断，缺乏数据验证，效果难以评估。
· 指标若设计不当，易异化为考核工具，反而抑制协作。

AFTER · AI 智能赋能升级

· **统一度量**：交付周期、需求吞吐、缺陷率、变更失败率等端到端指标自动采集，口径一致、真实可溯。
· **全景可视**：跨工具数据自动汇聚为研发效能看板，瓶颈环节一目了然，无需人工汇总。
· **AI 归因与预警**： AI 自动分析指标波动，归因到具体环节与变更，并提前预警交付风险，改进有据可依。
· **改进导向**：指标服务于改进而非考核，通过团队数据确认改进方向，避免指标滥用。

度量是为了让改进有据可依，而非用于考核加压——看见问题，才能解决问题

研发效能可视可测，瓶颈定位与改进验证有数据支撑。

研发体系建设 *问题跟踪*

BEFORE · 传统模式痛点

- 缺陷与需求分散在不同渠道，登记、分派依赖人工，易遗漏、易滞后。
- 问题定位依赖人工关联代码与日志，根因追溯周期长。
- 修复完成后缺少统一的验证与复盘，同类问题反复出现。
- 处理过程产生的经验散落各处，未沉淀为团队可复用的资产。

AFTER · AI 智能赋能升级

- **统一闭环**：缺陷、任务、需求全量纳入同一跟踪体系，状态流转透明，无人工值守环节亦不遗漏。
- **AI 智能分诊**： AI 依据问题描述自动分类、定级并分派至对应负责人，缩短响应与流转时间。
- **根因联动**：问题与代码提交、构建记录自动关联， AI 辅助定位引入变更，修复与验证有据可依。
- **知识回流**：闭环后的问题自动沉淀为回归用例与经验知识，反哺研发各环节，同类问题不再重复发生。

每一次问题闭环，都是一次向质量基线与知识库的回流——闭环即沉淀

问题闭环周期缩短，同类缺陷复发率下降，处理经验沉淀为组织资产。

能力平台与场景落地



能力平台与场景落地 *AI 能力中台*

统一汇聚、封装与运营 AI 能力的平台底座，是能力从 " 项目制 " 走向 " 平台制 " 的载体。

1. 能力汇聚：将散落在各项目中的 AI 能力统一纳管，避免重复建设与各自为政。
2. 原子封装：将文档解析（ OCR ）、图像质检、时序预测、流程自动化（ RPA ）等能力封装为标准接口。
3. 统一调用：业务与研发系统经标准入口接入，统一鉴权、配额与计量，无需各自对接模型。
4. 运营观测：对调用量、效果与成本统一度量，为能力迭代与资源投入提供依据。

实现 " 一次建设、多处复用 " ，使研发团队获得接入 AI 的标准通道，显著降低 AI 使用门槛。

能力平台与场景落地 *通用模型与工具能力*

为上层场景提供统一的模型引擎与公共工具，是能力中台在大模型时代的支撑层。

1. 大模型网关：兼容本地私有化模型与第三方 API 模型，统一接入、统一管理调用。
2. 模型路由与升级：按场景选择与切换模型，模型迭代对上层透明，避免技术锁定。
3. 知识库与上下文管理：提供统一的知识检索与上下文工程工具，支撑场景的精准调用。
4. 合规与安全管控：敏感数据不出域，调用全程留痕审计，满足数据安全与 GMP 相关要求。
5. 成本管控：按场景、按用量统一计量与配额管理，模型成本可预期、可优化。

模型能力随取随用、可按需替换而不被锁定，并在合规与成本的双重约束下稳定运行。

能力平台与场景落地 *代码助手*

把第一部分编码环节已验证的 AI 提效，固化为全员统一使用的组织级工具。

1. 组织级接入：统一接入开发环境，覆盖代码补全、生成、解释与重构，替代个人零散使用。
2. 审查与测试联动：与代码审查、单元测试生成相结合，承接质量门禁中的 AI 审查能力。
3. 规范与知识内嵌：内置团队编码规范与私有项目知识，生成结果天然符合组织标准。
4. 安全管控：代码在受控环境内处理，使用行为留痕，兼顾效率与代码资产安全。

让每位开发者处于同一提效水平线上，编码提效由 " 个人技巧 " 沉淀为 " 组织能力 " 。

能力平台与场景落地 *智能运维*

覆盖系统上线之后的运行环节，承接研发支撑体系在交付后的保障闭环。

1. 智能告警：对运行异常自动发现并收敛告警，减少误报与漏报。
2. 日志与根因分析：对日志与调用链自动分析，辅助根因定位，缩短排障时间。
3. 容量与健康预测：基于运行数据预测容量水位与故障风险，由被动处置转为主动预防。
4. 与发布流程衔接：与 CI/CD 及发布策略贯通，系统上线前后的状态连贯可观测。

线上问题早发现、快定位，保障交付后的持续稳定运行，为研发效能闭环兜底。

能力平台与场景落地 *需求智能分析*

把住需求进入开发前的入口，回扣第一部分需求梳理环节的提效能力。

1. 结构化梳理：将自然语言描述的需求转为结构化需求单，口径统一、字段完整。
2. 影响面分析：关联代码、接口与数据资产，评估需求改动的影响范围。
3. 工作量估算：结合历史数据给出工作量与排期建议，为计划安排提供依据。
4. 交付前置准备：同步生成验收标准与用例建议，为测试环节提供输入。

从源头降低需求的模糊与带病进入，提升后续各环节的交付确定性。

实施路径与保障机制

分阶段落地计划

建立组织保障

人才培训与培养

实施路径与保障机制 *分阶段落地计划*

把六环节提效、支撑体系与能力平台组织为有序推进的节奏，分步验证、降低风险、快速见效。

1. 试点验证阶段：选择 1-2 个真实需求，以 AI 全流程走通需求梳理至文档交付，验证提效幅度，沉淀样板案例与工具使用规范，时间以月计。

2. 体系成型阶段：建设 CI/CD 流水线、质量门禁、研效度量与问题跟踪支撑体系，规模化推广代码助手等已验证工具，形成组织级规范。

3. 平台与规模化阶段：上线 AI 能力中台与通用模型能力，代码助手、智能运维、需求智能分析全面落地，团队能力与平台同步成型，时间跨度以月计、留弹性。

每个阶段有明确重点、交付物与验收标准，能力按节奏沉淀，不追求一步到位。

实施路径与保障机制 *建立组织保障*

使 AI 研发效能建设不依赖个别人员与临时项目，而以组织、制度与治理机制持续运转。

1. 推进组织：成立由 IT 负责人牵头的专项推进组织，明确决策、协调与跨部门衔接机制。
2. 制度流程：将 AI 使用规范、代码与数据安全边界制度化，纳入日常开发流程。
3. 治理运营：质量门禁、度量口径、审计留痕等治理机制有明确的责任主体持续运营，而非一次性建设。
4. 资产归属：合作模式与交付物（代码、文档、模型）归属事先明确，避免供应商绑定与资产流失。

权责清晰、机制常态，数字化能力沉淀为企业资产，而非个人或项目资产。

实施路径与保障机制 *人才培训与培养*

AI 工具可以快速引入，判断与决策能力必须由自有团队掌握——围绕 AI 打造一支能独立判断、持续交付的 IT 队伍。

1. 分层培养：按工具使用、输出判断、业务结合、战略决策四个层次规划成长路径，逐级进阶。

2. 边干边学：以真实商业化项目为载体，由资深人员担任导师带教，在项目中练中学，而非书本式培训。

3. 判断力是根基： AI 输出须经人工校验，团队须掌握 " 该信什么、该改什么、什么不能交给 AI" 的边界—— AI 很强大，但不能完全依赖，判断与决策的根基始终在自有团队。

4. 成果归属：合作共建期间，双方产出的代码与文档等成果均归康希诺所有，团队能力与企业资产同步沉淀。

自有团队具备以 AI 放大自身能力并独立判断的根基，一支强有力的开发团队成为企业的长期资产。

案例

THANKS